

Adaptive Coding System Prototype: State of Developer Ecosystem Report 2025

Reahlyn Ermita, Jay Mark Labial, Jim Margaja, Jake Quejada, Jude Rabaya

Introduction

The goal of this activity is to design an adaptive coding system that changes its hints, feedback, debugging help, code completion and AI assistance according to different types of users. The system uses a user model built from real survey data, so each user gets the kind of help their group actually needs.

Investigation

We used the publicly available [JetBrains State of Developer Ecosystem 2025](#) raw dataset (developer_ecosystem_2025_external.csv), with 24,534 developer responses collected from April to June 2025.

Data source

Item	Details
Dataset	JetBrains State of Developer Ecosystem 2025 , anonymized raw data
Collected	April to June 2025
Respondents	24,534 developers from 194 countries and regions
Raw size	4,740 columns (one per question or answer option)
Personal data	None; the dataset is anonymized

Characteristics investigated. From the 4,740 columns, 23 were kept:

Area	Columns	What it measures
Programming experience	code_yrs, employment_status	Years of professional coding; whether the respondent is a student
Development environment	ide_main, main_lang	Main IDE or editor; main programming language

Area	Columns	What it measures
AI-assisted programming	ai_tool_* (6 tools), uses_ai_tools	Which AI tools they regularly use (ChatGPT, GitHub Copilot, JetBrains AI Assistant, Cursor, Claude, Gemini)
AI-assisted programming	ai_task_freq: * (6 tasks)	How often they use AI for code completion, generating code, explaining code, debugging, explaining bugs and learning
Feature importance	ai_importance: * (4 features)	How important completion quality, chat answer quality, an easy interface and IDE integration are

Data cleaning. The text was loaded as UTF-8 so answers such as "3–5 years" display correctly. code_yrs was grouped into three levels: **Beginner** (0–2 years, 5,836 people), **Intermediate** (3–10 years, 10,482) and **Experienced** (11+ years, 8,216). AI tool columns were converted to Yes/No. No rows were deleted; blank answers were excluded question by question, because several AI questions were shown only to AI users (about 7,000 people).

Data Analysis

We compared the three experience groups on every measure and kept the ones with a clear difference between Beginners and Experienced developers. Percentages are the share of each group; the last column is Beginner minus Experienced, in percentage points.

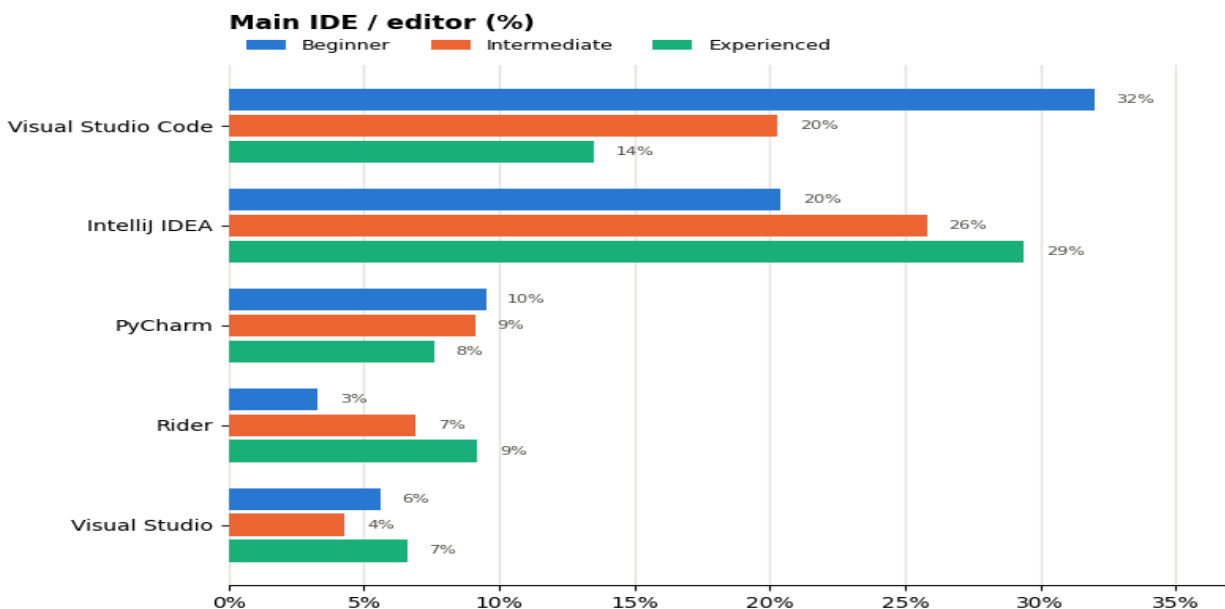
Measure	Beginner	Intermediate	Experienced	Difference
Use AI often to explain code	59%	45%	34%	+25
Use AI often to explain bugs and fixes	64%	52%	39%	+25
Use AI often for debugging	50%	39%	27%	+23
Use AI often to learn about coding	62%	54%	45%	+17
Use ChatGPT (AI users)	63%	52%	44%	+19
Main editor is VS Code	32%	20%	14%	+18
Main language is Python	27%	18%	13%	+14
Use AI code completion often	75%	79%	82%	−7

Rate completion quality "Very important"	63%	71%	76%	-13
Use GitHub Copilot (AI users)	31%	38%	44%	-13
Use JetBrains AI Assistant (AI users)	11%	20%	28%	-17
Use AI often to generate code	74%			

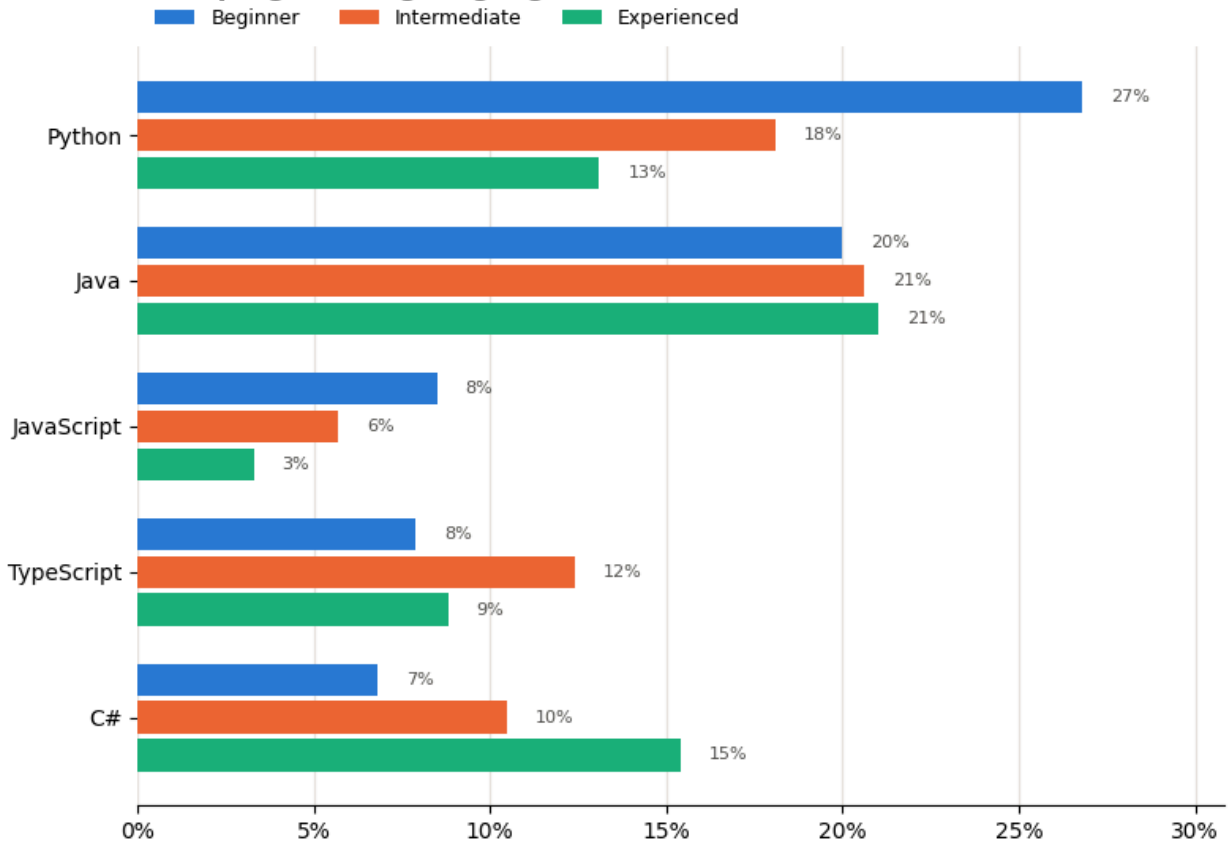
Key findings

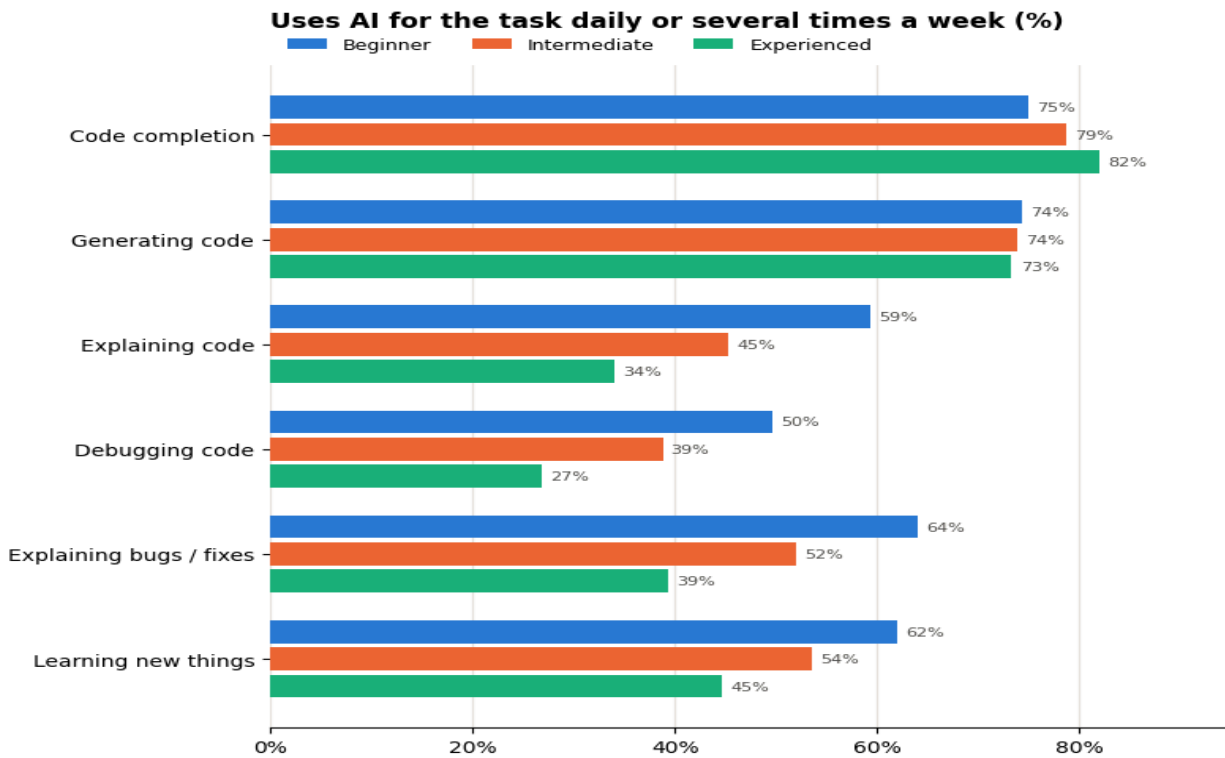
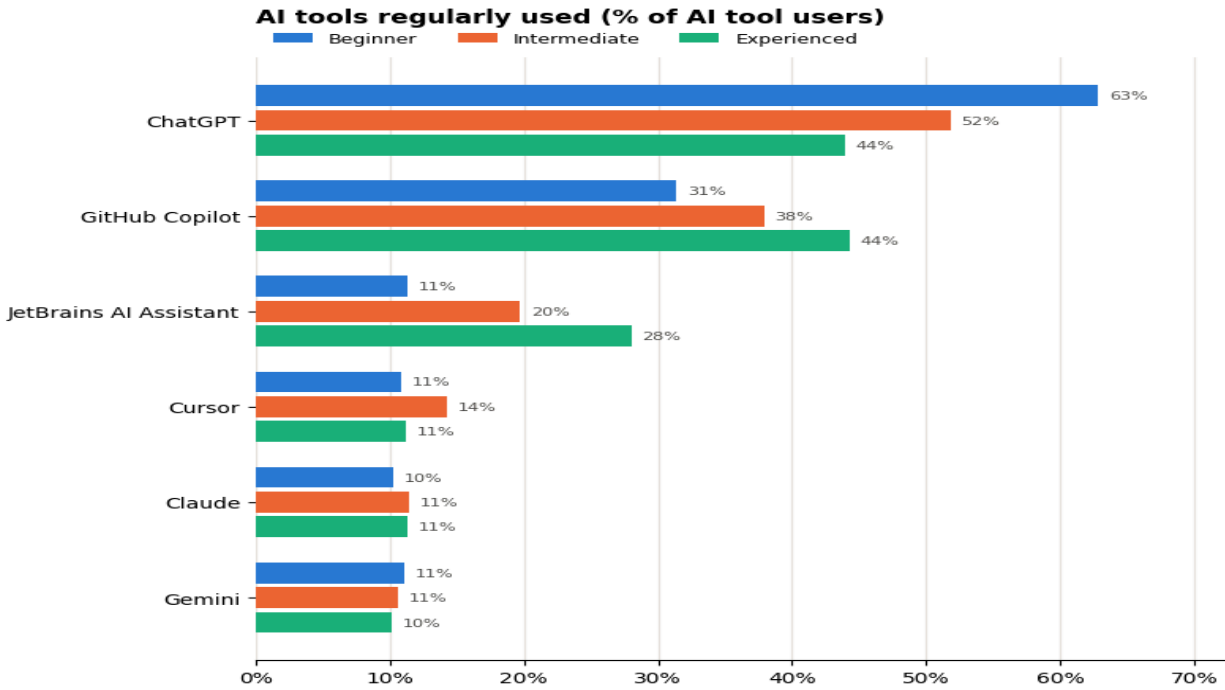
The groups differ most in **why** they use AI, not in whether they use it. About 80% of every group uses at least one AI tool.

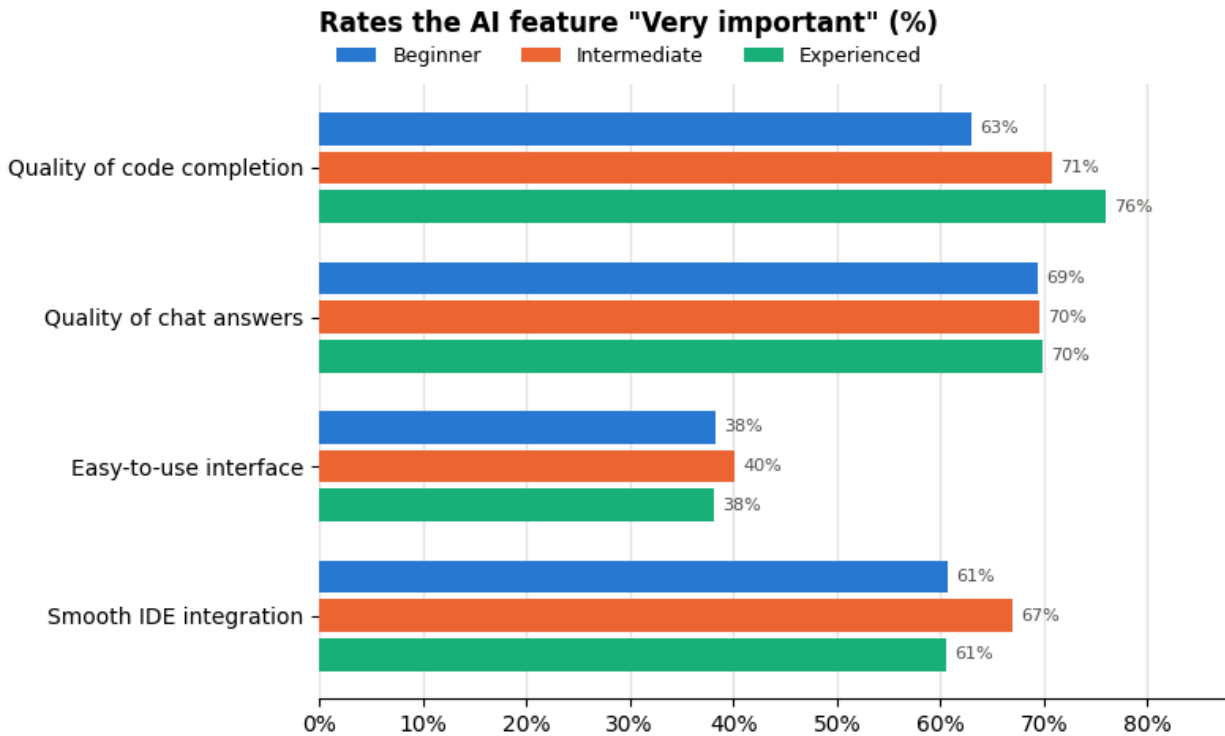
1. There is a clear difference between beginners (**0–2 years**) and experienced developers (**11+ years**) in **why** they use AI.
2. **Beginners** use AI to understand code: **59%** often use it to **explain code (vs 34% of experienced developers)**, **64%** to explain bugs (vs 39%) and **50%** to debug (vs 27%).
3. **Experienced** developers use AI for **speed**: **82%** often use AI code completion (vs 75%), and 76% rate completion quality "Very important" (vs 63%).
4. **Beginners prefer chat tools like ChatGPT** (63% vs 44%); **Experienced developers** prefer AI inside the IDE, like GitHub Copilot (44% vs 31%) and JetBrains AI Assistant (28% vs 11%).
5. **Beginners use lighter editors** (VS Code 32% vs 14%) and more often **use AI to learn** (62% vs 45%).
6. Most developers already use AI tools: about 80% of every group uses at least one.



Main programming language (%)







Analysis and User Profiles

The analysis compared the three experience groups on every measure and kept the measures with a clear gap between Beginners and Experienced developers. The table below shows those gaps; the last column is Beginner minus Experienced, in percentage points.

Measure	Beginner	Intermediate	Experienced	Gap
Use AI often to explain code	59%	45%	34%	+25
Use AI often to explain bugs and suggest fixes	64%	52%	39%	+25
Use AI often for debugging	50%	39%	27%	+23
Use AI often to learn about coding	62%	54%	45%	+17
Use ChatGPT (AI users)	63%	52%	44%	+19

Measure	Beginner	Intermediate	Experienced	Gap
Main editor is VS Code	32%	20%	14%	+18
Main language is Python	27%	18%	13%	+14
Use AI code completion often	75%	79%	82%	-7
Rate completion quality "Very important"	63%	71%	76%	-13
Use GitHub Copilot (AI users)	31%	38%	44%	-13
Use JetBrains AI Assistant (AI users)	11%	20%	28%	-17
Use AI often to generate code	74%	74%	73%	+1

Positive gaps describe what Beginners need more: explanation, debugging help and learning support, delivered through a chat. Negative gaps describe what Experienced developers need more: fast, high-quality completion inside the IDE. Intermediate developers fall between the two on every measure, so the lab focuses on the two ends as Profile A and Profile B.

Profile A: Beginner Learner

- Low programming experience (0–2 years); 36% are students
- Codes mainly in Python, in a lightweight editor such as VS Code
- Needs high support: step-by-step hints, detailed error explanations and guided debugging
- Uses AI mainly to explain code, fix bugs and learn concepts, through a chat tool

Profile B: Experienced Professional

- High experience (11+ years)
- Codes mainly in Java or C#, in a full IDE such as IntelliJ IDEA or Rider
- Prefers minimal traditional assistance: no hints, short error messages, quick fixes
- Uses AI mainly for fast code completion, built into the editor

User model

The system uses the following attributes:

Attribute	Profile A: Beginner	Profile B: Experienced
Experience Level	Beginner	Experienced
Years Coding	0-2 years	11+ years
Preferred Language	Python	Java/C#
Editor Style	Lightweight(VS CODE)	Full IDE (IntelliJ)
Guidance Level	HIGH	LOW
Error Details	DETAILED	COMPACT
Debugging Assistance	HIGH	LOW
Code Completion	MODERATE	HIGH
AI Assistance	CHAT TUTOR	INLINE(inside the editor)
Learning Support	ON	OFF

Prototype Implementation

The prototype is a Google Colab notebook (CS412_Lab2_Adaptive_Coding.ipynb) written in Python, with an interactive page built in HTML, CSS and JavaScript and shown inside Colab.

Notebook structure

Part	What it does
Part 1 – Investigate	Loads the cleaned Excel file and groups users into Beginner, Intermediate and Experienced
Part 2 – Analyze	Builds 6 comparison tables and 5 charts (pandas, matplotlib)
Part 3 – User model	Stores the user models as Python dictionaries (USER_MODELS)

Part 4 – Adaptation rules	Stores the rules and the adapt() function, with evidence for each rule
Part 5 – Python feedback demo	Checks the same buggy code with Python's real compiler and prints different feedback per model
Interactive prototype	The Adaptive Coding System Prototype page (HTML shown with IPython.display)

The interactive prototype includes:

- Profile switcher: Profile A · Beginner and Profile B · Experienced buttons
- Task panel: the coding task and step-by-step hints (Beginner only)
- Code editor: line numbers, with error lines highlighted in red
- Toolbar: Check code, Generate code (AI) and Reset
- Suggestion bar: keyword or full-line AI suggestions, depending on the profile
- Problems area: detailed error cards or one-line errors, with the matching fix buttons
- Active User Model panel: shows all 9 attributes of the selected profile
- AI Assistant: Chat Tutor (Beginner) or Inline AI (Experienced), both simulated
- Micro-Learning Concept: a short lesson (Beginner only)
- Adaptation Evidence & Signals: 8 cards with bars showing the survey percentages behind each rule

How it works: when a profile is selected, the page reads that profile's user model and applies the rules above. Check code scans every line of the code for a missing colon and unclosed brackets. Both profiles start from the same starter code with 2 bugs (a missing : on line 1 and a missing) on line 8), so their results can be compared directly.

Demonstration

Both profiles were tested with the same code and the same steps: select the profile, press Check code, then fix the errors.

Test 1 – Profile A (Beginner)

- Who is this user? A Beginner with 0–2 years of experience who codes in Python in a lightweight editor and uses a chat AI to understand code.
- What does the user model contain? Guidance High, Error detail Detailed, Debugging High, Completion Moderate, AI mode Chat tutor, Learning On.
- How does the system adapt?

What happened:

- All 4 hints are shown as steps.
- Both bugs are shown as detailed red cards (Line 1: Missing Colon, Line 8: Bracket Never Closed) with "What happened" and "How to fix".
- Each card has debug steps and a "Still stuck? Show the fixed line" button, then "Apply this fix".
- The AI Tutor opens automatically: "I found 2 problems... Try fixing the first one yourself!"
- A micro-lesson on functions is shown.
- After both fixes: "No errors. Great job! Output: Average: 85.0".
- Why does it adapt this way? Beginners use AI far more to explain code (59%), explain bugs (64%), debug (50%) and learn (62%), mostly through ChatGPT (63%).

Adaptive Coding System Prototype

State of Developer Ecosystem Report 2025 (n = 24,534)

Profile A · Beginner

Profile B · Experienced

TASK

Write `average(scores)` that returns the average of a list, then print the average of [85, 92, 78]. The starter code has 2 bugs.

Step 1: Start the function with `def average(scores):` and keep the colon at the end.

Step 2: Inside the function, set `total = 0`, then add each score with a for loop.

Step 3: Return `total / len(scores)`.

Step 4: Call the function and print the result. Every (needs a matching).

Check code

Generate code (AI)

Reset

main.py

```

1 def average(scores)
2     total = 0
3     for s in scores:
4         total = total + s
5     return total / len(scores)
6
7 scores = [85, 92, 78]
8 print("Average:", average(scores))
9

```

Keyword suggestions appear after 3 letters

PROBLEMS EXPLAINED

Line 1: Missing Colon (:)

Your code:

`def average(scores)`

What happened:

Lines that start a block (def, if, for, while) must end with a colon.

How to fix:

Add : at the end of the line.

1. Go to line 1.

2. Compare it with the hint steps above.

3. Try to fix it yourself, then press Check code.

Still stuck? Show the fixed line

Line 8: Bracket Never Closed

Your code:

`print("Average:", average(scores))`

What happened:

Every opening bracket (needs a closing bracket).

How to fix:

Add) at the end of the line.

1. Go to line 8.

2. Compare it with the hint steps above.

3. Try to fix it yourself, then press Check code.

Still stuck? Show the fixed line

Active User Model

Rule Engine

Experience level

Beginner (0-2 yrs)

Language

Python

Editor Style

Lightweight

Guidance Level

High

Error Detail

Detailed

Debugging Assistance

High

Code Completion

Moderate

AI assistance mode

Chat tutor

Learning support

On

AI Assistant

Chat Tutor

Explain current code

Why is it wrong?

How do I fix this?

Teach me the concept

AI Assistant: I found 2 problems:

- Line 1: Missing Colon (:). Lines that start a block (def, if, for, while) must end with a colon.
- Line 8: Bracket Never Closed. Every opening bracket (needs a closing bracket).

Try fixing the first one yourself!

Micro-Learning Concept

Function Definition (def):

In Python, functions are declared with the def keyword. They need parenthesized parameters and must end with a colon (:) to start the indented block.

ADAPTATION EVIDENCE & SIGNALS (BEGINNER VS EXPERIENCED, FROM THE SURVEY)

Guidance & Tutorials

Uses AI to explain code

Beginner

59%

Experienced

34%

Error Details

Uses AI to explain bugs/fixes

Beginner

64%

Experienced

39%

Debugging Assistance

Uses AI for debugging

Beginner

50%

Experienced

27%

Code Completion

Rated feature "Very important"

Beginner

63%

Experienced

76%

AI Chat Tutor

AI users who use ChatGPT

Beginner

63%

Experienced

44%

Inline AI in the IDE

AI users who use GitHub Copilot

Beginner

31%

Experienced

44%

Micro-Learning

Uses AI to learn about coding

Beginner

62%

Experienced

45%

Generate Code (same for all)

Uses AI to generate code

Beginner

74%

Experienced

73%

Test 2 – Profile B (Experienced)

- Who is this user? an Experienced Professional with 11+ years of experience who works in a full IDE and uses AI inside the editor to code faster.

- What does the user model contain? Guidance Low, Error detail Compact, Debugging Low, Completion High, AI mode Inline, Learning Off.
- How does the system adapt?

What happened:

- No hints, no chat tutor and no lesson are shown.
 - Errors are one line each: 1:20 SyntaxError: expected ':' and 8:6 SyntaxError: '(' was never closed.
 - Each error has a one-click Quick fix and an inline AI suggestion of the corrected line.
 - Typing on line 1 shows a full-line AI suggestion: `def average(scores):`.
 - After the fixes: "0 problems · Average: 85.0".
- Why does it adapt this way? experienced developers rarely use AI to explain code (34%) or debug (27%), but they value fast code completion (76% "Very important") and use AI inside the IDE (Copilot 44%).

Adaptive Coding System Prototype
State of Developer Ecosystem Report 2025 (n = 24,534)

Profile A · Beginner
Profile B · Experienced

TASK
Write `average(scores)` that returns the average of a list, then print the average of [85, 92, 78]. The starter code has 2 bugs.

Check code
Generate code (AI)
Reset

main.py

```

1 def average(scores)
2     total = 0
3     for s in scores:
4         total = total + s
5     return total / len(scores)
6
7 scores = [85, 92, 78]
8 print("Average:", average(scores))
9

```

Suggestions: AI: `def average(scores):`

PROBLEMS
1:20 SyntaxError: expected ':' Quick fix AI: `def average(scores):`
8:6 SyntaxError: '(' was never closed Quick fix AI: `print("Average:", average(scores))`

Active User Model
Rule Engine

Experience level Experienced (11+ yrs)
Language Java
Editor Style Full IDE
Guidance Level Low
Error Detail Compact
Debugging Assistance Low
Code Completion High
AI assistance mode Inline
Learning support Off

AI Assistant
Inline

AI suggestions appear inside the editor and next to each error. No chat, fewer interruptions.

ADAPTATION EVIDENCE & SIGNALS (BEGINNER VS EXPERIENCED, FROM THE SURVEY)

Guidance & Tutorials
Uses AI to explain code
Beginner 59%
Experienced 34%

Error Details
Uses AI to explain bugs/fixes
Beginner 64%
Experienced 39%

Debugging Assistance
Uses AI for debugging
Beginner 50%
Experienced 27%

Code Completion
Rated feature "Very important"
Beginner 63%
Experienced 76%

AI Chat Tutor
AI users who use ChatGPT
Beginner 63%
Experienced 44%

Inline AI in the IDE
AI users who use GitHub Copilot
Beginner 31%
Experienced 44%

Micro-Learning
Uses AI to learn about coding
Beginner 62%
Experienced 45%

Generate Code (same for all)
Uses AI to generate code
Beginner 74%
Experienced 73%

Result: both users fix the same code and get the same output (Average: 85.0), but the system gives each one different help. This clearly shows that the system does not behave the same way for every user.

Reflection

Which user characteristics did you model, and why did you select them?

We modeled nine attributes: experience level, preferred language, editor style, guidance level, error detail, debugging assistance, code completion, AI assistance mode and learning support. We selected them because they showed the largest differences between Beginners and Experienced developers in the survey (up to 25 percentage points), and because each one can directly change what a coding tool shows. Characteristics with no difference, such as using AI to generate code (74% vs 73%), were not modeled as adaptive and stayed the same for everyone.

What patterns in the data supported your user model?

- Explaining code (59% vs 34%), explaining bugs (64% vs 39%), debugging (50% vs 27%) and learning (62% vs 45%) all go down as experience goes up, so Beginners get high guidance, detailed errors, guided debugging and lessons.
- Code completion goes up with experience (75% vs 82% use it often; 63% vs 76% rate it "Very important"), so Experienced developers get eager, full-line completion.
- Beginners prefer chat tools (ChatGPT 63% vs 44%), while Experienced developers prefer AI inside the IDE (GitHub Copilot 31% vs 44%, JetBrains AI 11% vs 28%), which gave us the Chat tutor vs Inline AI modes.
- Beginners use lighter editors (VS Code 32% vs 14%) and more often use Python (27% vs 13%), which shaped the editor style and language values.

How did the user model influence the behavior of your system?

The same code and task produce two different experiences. For the Beginner model, the system shows 4 hints, detailed error cards, step-by-step debugging, an AI Chat Tutor and a micro-lesson. For the Experienced model, it hides the hints, tutor and lesson, shows one-line errors with a Quick fix, and gives inline AI and full-line completion. Every one of these differences comes from a single attribute in the user model, so changing one value changes only that behavior.

What limitations does your user model have?

- The survey is self-reported, so it shows what developers say they do, not what they actually do.
- Most respondents are professionals; students are only 36% of the Beginner group, so USTeP students may differ.
- Several AI questions were answered by only about 7,000 AI users, and the dataset cannot separate non-AI users from people who skipped the question.
- Users are grouped by years of experience only, and the model is static: it does not change as a beginner improves.

- The dataset had no questions on formatting or syntax highlighting, so those features were not adapted.
- The AI assistant in the prototype is simulated, not a real AI.

What additional information could make the user model more useful?

- Real usage data from USTeP, such as the errors a student makes, how many hints they open and how long they take to fix a bug, so the model can update itself over time.
- A short survey of USTeP students about their own preferences, including formatting, syntax highlighting and how much AI help they want.
- The type and difficulty of each task, since even a skilled student may want hints on a new topic.

Conclusion

The prototype successfully demonstrates an adaptive coding environment. Using survey data from 24,534 developers, we found that beginners use AI mainly to understand code, while experienced developers use it mainly for speed. We turned this difference into two user models and seven adaptation rules, so the same coding task gives beginners guidance and explanations and gives experienced developers fast, minimal help.